



AUTONOMOUS AGENTS AND MULTIAGENT SYSTEMS

The project aims to explore several components of Multi-Agent System (MAS), including coordination, decision-making, and learning.

Students are free to choose the programming language to develop their system.

PROJECTS

We can have 2 types of projects focusing on:

- the reasoning of the agents (game-theory or multi-agent learning);
- orchestration, coordination or collaboration between agents;

For any of these options, you should identify a scientific paper or existing work and reproduce it.

Reproducing is neither a complete reproduction (might be too difficult or too extensive) nor restricting variations that you might be motivated to introduce. It should nevertheless be focused on the topics of the course.

For instance, developing a multi-agent game is not relevant for this course, but using a game you developed in another course and studying the multi-agent interactions or learning is relevant here.

EVALUATION CRITERIA

Grading will be based on the following criteria:

- description of the problem;
- identification of limitations of the work;
- critical comparison between the implementation alternatives included advantages and disadvantages;
- identification of limitations of the solutions provided and possible solutions;
- identification of limitations of the results and simulations;

Your implementation does not need to be perfect, but you should identify how you could improve it, and what was done due to some practical limitations.

ORGANIZATION

GROUP FORMATION

Groups of 3, inclusive for student-workers.

You can change campus.

All students should be able to explain everything that is submitted. Even if you divide the work, you should still study everything.

PROJECT PROPOSAL PRESENTATION

On the week 2, you should make a 5 minutes presentation and submit it, that includes the plan for your project in the Fenix system.

It should include:

- An introduction that includes motivation, related work, problem definition, relevance, and objectives of the project;
- A description of the approach with the specification of the environment, (multi)agent system, and system architecture. This section should also include text to explain why the design choices are adequate to address the problem;
- A description of the empirical evaluation. Define a set of metrics that can validate the project's objectives.

INTERMEDIATE DELIVERY

On the week 4, you should make a 5 minutes presentation and submit it, with the initial results of your project. It should include a working prototype of your domain, some orchestration between the agents, or a mathematical model.

FINAL DELIVERY

On the week 7, you should make a 7 minutes presentation, and submit it with the final result of your project. Including a report.

The final delivery with the following elements:

1. **Full source code** and **executable** with a README that explains how to run the system
2. **Final report** (limit of 4 pages of text, plus figures and references, using the *ACM Primary Article Template* (2-column article)).

Due to public holidays, some labs will need to be rescheduled.

GRADING

Project Proposal Presentation 20% + Intermediate Delivery 20% + Final Delivery with Discussion 60%

PROJECT IDEAS

We will consider two main themes for projects. Game theory topics and agent orchestration.

You can suggest any other works. Discuss with you lab teacher to ensure that the difficulty is similar to the others.

AGENT REASONING IN MULTI-AGENT SYSTEMS

In this track, we want you to analyse the behavior of MAS, the reason for their decision, their adaptation, and study different algorithms for decision making, including learning or not.

1. A computational model for the question “Why do competitors open their stores next to one another?”
(https://www.youtube.com/watch?v=jLLgxeNBK_8&ab_channel=TED-Ed)
2. Ad Hoc Autonomous Agent Teams: Collaboration without Pre-Coordination (change to a computational one)
(<https://www.cs.utexas.edu/~pstone/Papers/bib2html/b2hd-AAA10-adhoc.html>)
3. Modelling how cleaner fish approach an ephemeral reward task demonstrates a role for ecologically tuned chunking in the evolution of advanced cognition
(<https://journals.plos.org/plosbiology/article?id=10.1371/journal.pbio.3001519>)

AGENT ORCHESTRATION

In this track, we want you to focus more on the generation of behavior. We need to include multiple agents, including the user of not, consider how to orchestrate the agents, analyse the coordination and computational complexity. You should discuss (and use if useful) some of the different current frameworks and methodologies, e.g. crewAI, MCP, RAG, ReAct (Reason + Act), Reflection / Self-Correction, Multi-Agent Orchestration, Stateful Graphs, Flow Engineering/Agentic SDLC.

1. Create a team of agents that make a "Choose Your Own Adventure" or DND type game. You can have a game master and some agents as players, allowing a person to also play the game together. Or you can have a team of agents developing the game.
(D. Chen, H. Wang, Y. Huo, Y. Li, and H. Zhang, “Gamegpt: Multi-agent collaborative framework for game development,” 2023. <https://arxiv.org/abs/2310.08067>)
(<https://llmarena.com/>)
2. Captain Sonar AI First Mate (Gaming Agency) - Develop an agent for the Captain Sonar boardgame. The game is not difficult to model using AI assistant, and can be used to explore both planning algorithms, coordination and learning.
(<https://boardgamegeek.com/boardgame/171131/captain-sonar>)
3. Autonomous Code Debugger (Self-Healing Code) - An agent receives a Python error (stack trace), uses a "File System Tool" to read the code, explains the bug, and applies a fix. The agent needs to handle the interaction with a user requesting the correction.
(<https://github.com/OpenBMB/ChatDev>)
4. Deep Research & Reporting Agent (Iterative Search) - A group of agents that doesn't just "search" once, but performs "Recursive Search"—it searches, reads the summaries, identifies missing info, and

searches again until a research goal is met.

(<https://arxiv.org/abs/2504.03160>)

USE OF LLM SYSTEMS

Use of modern A.I. tools is strongly recommended. In this course, we want you to explore their efficient use, verify their limitations, and be aware of cost/privacy/safety/ethical issues involved.

Note that anything that is submitted is the responsibility of the students. Everything that is submitted should be understood by you.

During the discussion, it is not acceptable to answer a question with “ChatGPT told me to”. Any code or decision that the students cannot justify will imply a 0 (zero) on that component.

The use of paid versions will make it easy to create bigger and more spectacular systems. This will not be considered for grading purposes. Only the analysis of the system and the architecture.

There are some free A.I. resources available. Even paid services have a free tier that can be exploited.

Free A.I. resources <https://chat.iaedu.pt/>, <https://gemini.google/students/>, <https://ollama.com/>.

The report should include a description of the methodology used to integrate code developed by A.I.